

**UNIVERDAD DE SAN CARLOS DE GUATEMALA
CENTRO UNIVERSITARIO DE OCCIDENTE DIVISIÓN DE CIENCIAS DE LA
INGENIERÍA
LENGUAJES DE PROGRAMACION
PROYECTO 1
SEGUNDO SEMESTRE 2026
Fernan Andoni González Hernández 202331283**

MANUAL TECNICO

ANALIZADOR LEXICO PROMPTZAL

EMPRESA: ANTHROPIC

INSTITUCION: ingenieria en ciencias y sistemas

Fecha: 17/09/2026

TECNOLOGIAS UTILIZADAS

Tecnologia	uso
Java 21	Desarrollo del sistema y analizador lexico
NeatBeans	Entorno de desarrollo
Java Swing	Desarrollo de interfaz grafica
Graphviz	Generacion de la representacion grafica del AFD
GitHub	Control de versiones
Linux	Sistema Operativo

ESTRUCTURA DEL PROYECTO

El proyecto esta organizado mediante diferentes paquetes, separando el analisis lexico, los modelos de datos, la interfaz grafica y la generacion del automata.

La estructura principal del proyecto es la siguiente

```
PracticalLenguajes/  
├──  
├── src/  
├── main/  
├── java/  
│   ├── Analizador/  
│   │   └── AnalizadorLexico.java  
│   ├── Interfaz/  
│   │   ├── Vista.java  
│   │   └── Vista.form  
│   ├── modelos/  
│   │   ├── TipoToken.java  
│   │   ├── Token.java  
│   │   ├── ErrorLexico.java  
│   │   └── Reportes.java  
│   ├── graficos/  
│   │   └── GeneradorAFD.java  
├── com/mycompany/practicallenguajes/  
├── PracticalLenguajes.java  
├── resources/  
├── Captura desde 2026-09-04 23-17-40.png  
├── Imagen pegada.png  
└── PracticalLenguajes-1.0-SNAPSHOT.jar
```

Paquete analizador

Este paquete contiene la clase principal encargada de todo el análisis léxico.

Analizador Léxico

Es el responsable de recorrer carácter por carácter la entrada proporcionada por el usuario, formar lexemas, identificar su tipo de token y registrar los errores encontrados.

La clase mantiene la información relacionada con:

- 1. entrada que se está analizando.**
- 2. Posición actual dentro de la entrada**
- 3. Fila actual**
- 4. Columna actual**
- 5. Número de token**
- 6. Tokens reconocidos**
- 7. Errores léxicos encontrados**

Donde el análisis se realiza de la manera manual mediante las estructuras de control de Java sin utilizar generadores automáticos de analizadores léxicos.

Paquete de modelos

Este paquete contiene las clases que representan la información durante el análisis.

TipoToken.java

Contiene el Enum utilizado para representar los diferentes tipos de tokens reconocidos por el analizador.

Token.java

Representa un token reconocido, almacena la información como Número del token.

Lexema

Tipo token

Fila

Columna

ErrorLexico.java

Representa un Error encontrado durante el análisis. Almacena:

Lexema o caracter que produjo el error.

Tipo de error.

Fila

Columna

Reportes.java

Contiene los metodos encargados de generar los reportes HTML del sistema

Actualmente genera lo que son reportes de tipo

Reporte de tokens

Reporte de errores

Reporte de estadisticas

Los reporetos incluyen informacion obtenida directamente del analisis lexico y utilizan HTML Y CSS pra representar los resultados de manera organizada.

Paquete Interfaz

Contiene las clases realcionadas con la interfaz grafica de la aplicacion

Vista.java

Es la ventana pricipal del sistema. Utiliza componentes de java Swing para permitir al usaurio:

Escribir codigo PromptZal

Crear un nuevo archivo

Abrir archivos

Guardar archivos

Ejecutal el analisis lexico

Visualizar los tokens reconocidos

Visualizar los errores encontrados

Generar reportes

Generar y visualizar el AFD

Vista.form

Es el archivo utilizado por NetBeans para almacenar el diseño visual de la interfaz creada mediante el editor grafico.

Paquete Graficos

Contiene las funcionalidades relacionadas con la representacion grafica del automata.

GeneradorAFD.java

Esta clases contiene la logica necesaria para generar la representacion del Automata finito Determinista del PromptZal.

El generador produce un archivo.dot utilizado por Graphviz y posteriormente una imagen.png que representa graficamente el automata.

La imagen generada corresponde al AFD general definido para el lenguaje PromptZal y no depende del contenido particular del archivo .pz que se encuentra analizando.

Paquete com.mycompany.practica1Lenguajes

Contiene la clase principal de ejecucion del proyecto.

Es la clase principal generada por Maven y constituyen el punto de inicio de la aplicacion.

Desde la clase se inicia la ejecucion del programa y se puede acceder a la interfaz grafica principal.

Recursos

La carpeta de resources contiene recursos utilizados para la aplicacion, principalmente imagenes utilizadas dentro de la interfaz grafica.

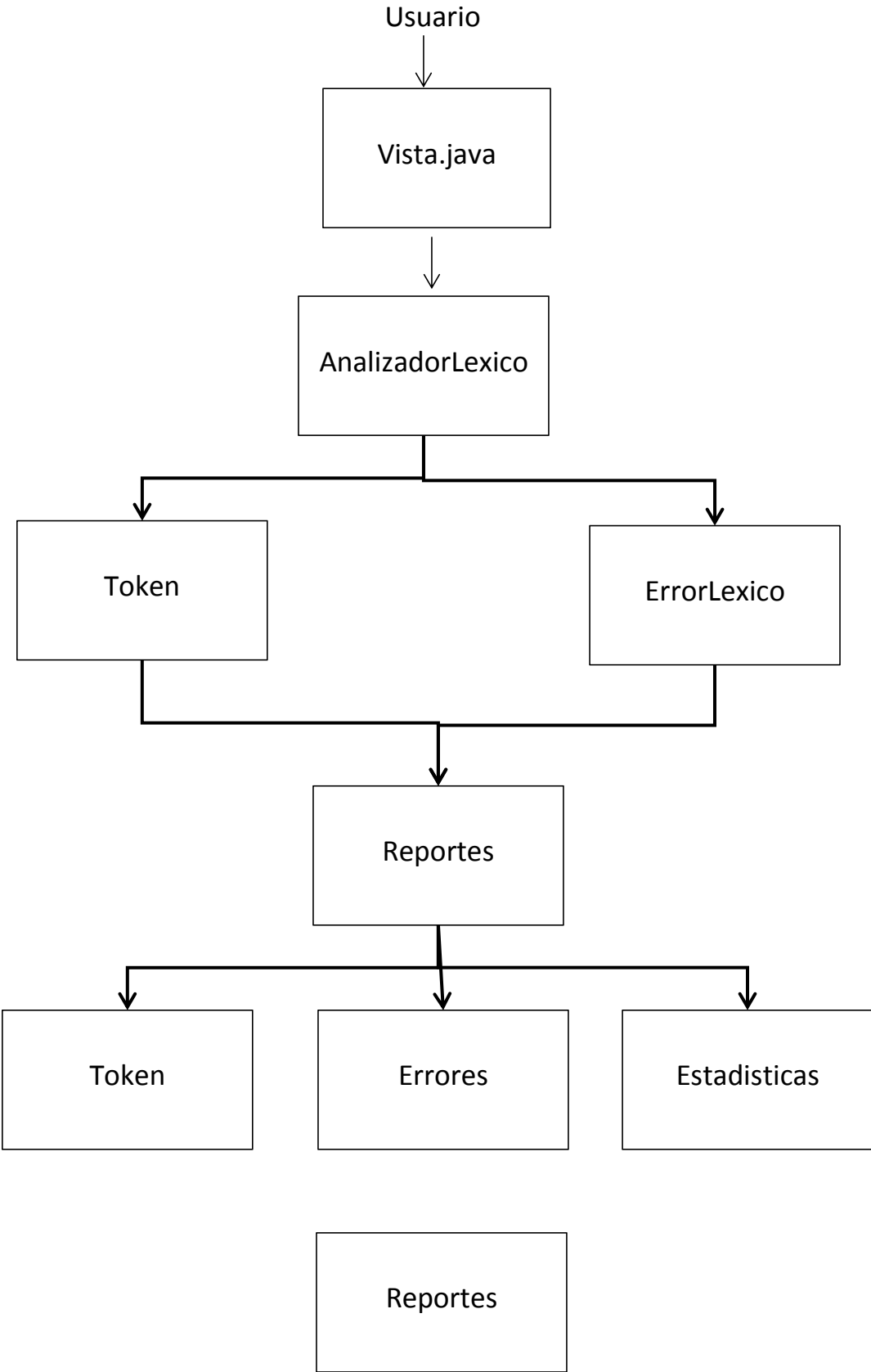
Archivo JAR

El proyecto tambien contiene el archivo.

Practica1Lenguajes-1.0-SNAPSHOT.jar

Este corresponde al archivo JAR generado a partir del proyecto y contienen la aplicacion compilada para su ejecucion.

Relacion general entre las clases



LISTA DE TOKENS DEL LENGUAJE

**DIRECTIVA,
RESERVADA,
COMANDO_IA,
FUNCION,
CONECTOR,
IDENTIFICADOR,
LITERAL_CADENA,
LITERAL_ENTERO,
LITERAL_DECIMAL,
OPERADOR_ASIGNACION,
OPERADOR_CONCATENACION,
DELIMITADOR**

AFD (Autómata Finito Determinista)

El Analizador Léxico PromptZal utiliza un Autómata Finito Determinista (AFD) como representación formal del proceso de reconocimiento de los elementos léxicos que forman parte del lenguaje.

El AFD permite representar el análisis mediante un conjunto finito de estados y transiciones. Cada transición se produce al recibir un determinado carácter o conjunto de caracteres de la entrada. De esta manera, el autómata permite representar las reglas utilizadas para determinar si una secuencia de caracteres corresponde a un elemento válido del lenguaje.

La representación del AFD se genera mediante la clase GeneradorAFD.java. Esta clase construye la información necesaria para generar un archivo con extensión .dot, el cual posteriormente es procesado mediante Graphviz para obtener una representación gráfica del autómata.

Funcionamiento general del AFD

El funcionamiento puede representarse de la siguiente manera:

El proceso comienza en un estado inicial. Conforme se reciben caracteres de la entrada, el autómata realiza transiciones entre estados de acuerdo con las reglas establecidas.

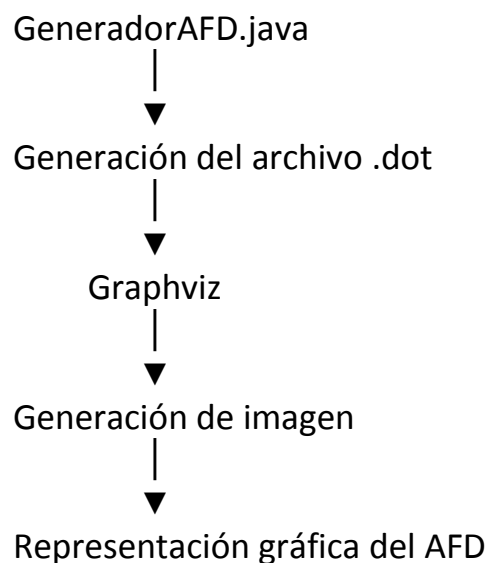
Cuando una secuencia cumple con las condiciones correspondientes, se alcanza un estado de aceptación. Esta información permite representar formalmente el reconocimiento de los elementos que posteriormente son utilizados por el analizador léxico.

Generación del AFD

La generación del AFD se realiza mediante la clase `GeneradorAFD.java`.

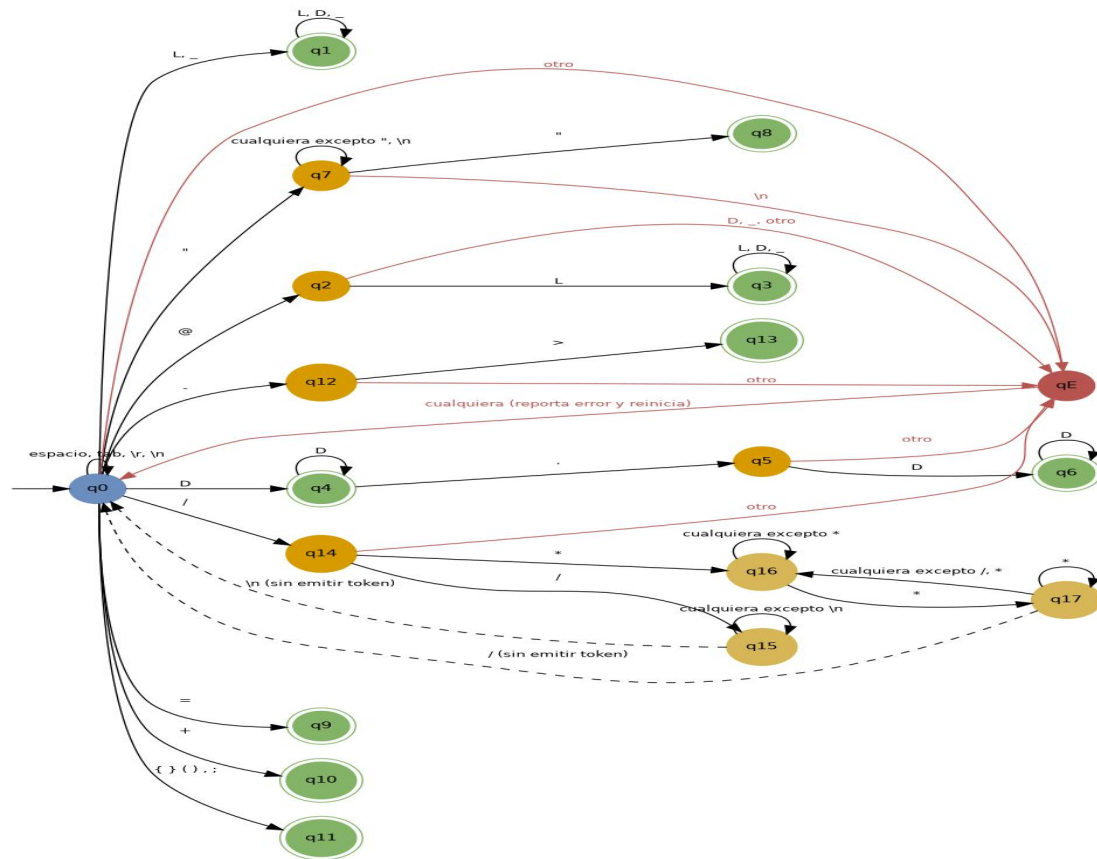
El proceso general es:

1. La aplicación ejecuta la funcionalidad encargada de generar el AFD.
2. La clase `GeneradorAFD` construye la representación del autómata.
3. Se genera un archivo `.dot`.
4. El archivo `.dot` es procesado mediante `Graphviz`.
5. `Graphviz` genera una imagen del autómata.
6. La imagen resultante puede ser visualizada desde la aplicación.



Representación gráfica

El sistema genera una representación gráfica del AFD correspondiente al lenguaje PromptZal.



La representación gráfica permite observar los diferentes estados y las transiciones existentes entre ellos. El estado inicial permite identificar el punto donde comienza el proceso de reconocimiento, mientras que los estados de aceptación representan las condiciones en las que una secuencia de caracteres puede considerarse válida según las reglas establecidas.

El AFD generado representa las reglas generales del lenguaje y no depende directamente del contenido específico del archivo .pz que se encuentre siendo analizado. Por esta razón, el mismo autómata puede utilizarse como representación general del lenguaje PromptZal.

Proceso de análisis léxico

El análisis léxico es realizado por la clase `AnalizadorLexico.java`. Su función principal consiste en recorrer la entrada proporcionada por el usuario y determinar los elementos que forman parte del lenguaje.

El análisis se realiza de manera manual utilizando estructuras de control propias de Java, sin utilizar generadores automáticos de analizadores léxicos.

Inicio del análisis

Cuando el usuario ejecuta el análisis, el contenido introducido en la interfaz es enviado al analizador léxico.

Antes de comenzar el recorrido se inicializan los datos necesarios para realizar el análisis, entre ellos:

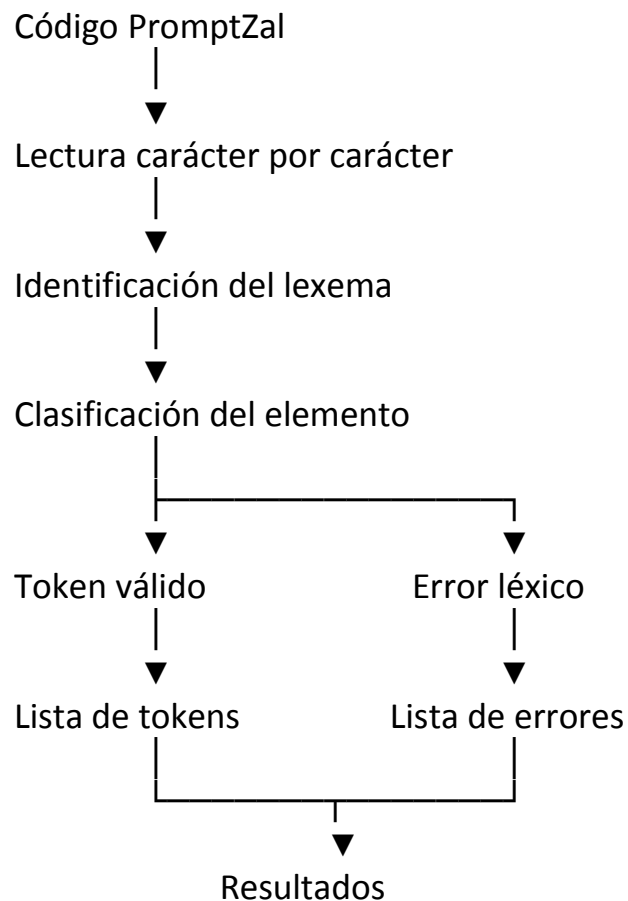
- 1.Posición actual.
2. Fila actual.
3. Columna actual.
4. Número de token.
5. Lista de tokens encontrados.
6. Lista de errores léxicos.

Recorrido de la entrada

El analizador recorre la entrada carácter por carácter.

Durante este proceso identifica qué tipo de elemento puede comenzar con el carácter actual y continúa leyendo los caracteres necesarios para formar el lexema correspondiente.

El proceso general es:



Reconocimiento de tokens

Cuando el analizador identifica correctamente una secuencia de caracteres, crea un objeto de tipo Token.

Cada token contiene la información necesaria para identificar el elemento reconocido.

Entre los datos almacenados se encuentran:

Número del token.

Lexema.

Tipo de token.

Fila.

Columna

Seguimiento de fila y columna

El analizador mantiene un control de la posición de cada elemento dentro del código fuente.

La fila permite identificar la línea donde se encuentra el elemento, mientras que la columna indica su posición dentro de dicha línea.

Este seguimiento es especialmente importante para el manejo de errores, ya que permite informar al usuario exactamente dónde fue encontrado un elemento que no cumple con las reglas del lenguaje.

Manejo de errores léxicos

El analizador también cuenta con un mecanismo para detectar y registrar errores durante el análisis.

Un error léxico ocurre cuando el analizador encuentra un carácter o una secuencia de caracteres que no puede ser reconocida como un elemento válido según las reglas implementadas para el lenguaje PromptZal.

Registro de errores

Cuando se detecta un error, se crea un objeto de la clase ErrorLexico.

Este objeto permite almacenar la información relacionada con el elemento que produjo el error.

La información registrada incluye:

- Lexema o carácter que produjo el error.

- Tipo de error.

- Fila donde ocurrió.

- Columna donde ocurrió.

De esta manera, el error no solamente se muestra al usuario, sino que queda registrado para poder ser utilizado posteriormente en el reporte de errores.

Continuación del análisis

El registro de un error permite conservar la información encontrada durante el análisis y continuar procesando el resto de la entrada cuando las reglas implementadas lo permiten.

Esto permite obtener una lista de los diferentes errores encontrados en un mismo archivo en lugar de depender únicamente del primer error encontrado.

Visualización de errores

Los errores detectados pueden ser consultados desde la interfaz gráfica de la aplicación.

La información mostrada permite al usuario identificar:

Qué elemento produjo el problema.

Qué tipo de error se produjo.

En qué fila ocurrió.

En qué columna se encuentra.

Codigo PromptZal

```
AGENTE redactor (  
  contexto = "Eres un redactor creativo"  
  
  variable tema = "Inteligencia artificial en Guatemala"  
  
  GENERAR artículo DESDE tema -> borrador  
  RESUMIR borrador EN 100 palabras -> resumen?  
  TRADUCIR resumen EN "ingles" -> final  
)  
  
EJECUTAR analista  
EJECUTAR redactor  
EXPORTAR salida, final & extra
```

Tokens

No.	Lexema	Tipo	Linea	Columna
1	@modelo	DIRECTIVA	1	1
2	"claude-sonnet-4-6"	LITERAL_CADENA	1	9
3	@rol	DIRECTIVA	2	1
4	"asistente de analisis"	LITERAL_CADENA	2	6
5	@formato	DIRECTIVA	3	1
6	"markdown"	LITERAL_CADENA	3	10
7	AGENTE	RESERVADA	11	1
8	analista	IDENTIFICADOR	11	8
9	(	DELIMITADOR	11	17
10	contexto	RESERVADA	12	3
11	=	OPERADOR ASIGNA...	12	12
12	"Eres un analista de ...	LITERAL_CADENA	12	14
13	variable	RESERVADA	14	3
14	ventas	IDENTIFICADOR	14	12

Errores

Lexema	Tipo de Error	Fila	Columna
%	Carácter no reconocido	15	22
#	Carácter no reconocido	17	69
\$	Carácter no reconocido	19	58
%	Carácter no reconocido	19	59
\$	Carácter no reconocido	19	60
"Eres un redactor creativo	Cadena sin cerrar	23	14
?	Carácter no reconocido	28	46
&	Carácter no reconocido	34	24

Reportes generados por el sistema

La clase Reportes.java contiene los métodos encargados de generar los reportes HTML obtenidos a partir de la información producida durante el análisis léxico.

Los reportes utilizan HTML y CSS para presentar la información de manera organizada.

Actualmente el sistema genera los siguientes reportes:

Reporte de tokens

Este reporte contiene la información de los tokens reconocidos durante el análisis.

Entre los datos mostrados se encuentran:

Número del token.

Lexema.

Tipo de token.

Fila

Columna.

Reporte de Tokens				
Tokens reconocidos por el analizador léxico de PromptZai				
Total de tokens encontrados: 84				
No.	Lexema	Tipo	Fila	Columna
1	@modelo	DIRECTIVA	1	1
2	"claude-sonnet-4-6"	LITERAL_CADENA	1	9
3	@rol	DIRECTIVA	2	1
4	"asistente de analisis"	LITERAL_CADENA	2	6
5	@formato	DIRECTIVA	3	1
6	"markdown"	LITERAL_CADENA	3	10
7	AGENTE	RESERVADA	11	1
8	analista	IDENTIFICADOR	11	8
9	{	DELIMITADOR	11	17
10	contexto	RESERVADA	12	3
11	=	OPERADOR_ASIGNACION	12	12
12	"Eres un analista de datos"	LITERAL_CADENA	12	14
13	variable	RESERVADA	14	2

Reporte de errores

Este reporte contiene los errores léxicos detectados durante el análisis.

La información permite identificar el elemento que produjo el error y su ubicación dentro del código.

Reporte de Errores Léxicos				
Errores encontrados durante el análisis léxico de PromptZal				
Total de errores encontrados: 8				
No.	Lexema	Tipo de error	Fila	Columna
1	%	Carácter no reconocido	15	22
2	#	Carácter no reconocido	17	69
3	\$	Carácter no reconocido	19	58
4	\$	Carácter no reconocido	19	59
5	\$	Carácter no reconocido	19	60
6	"Eres un redactor creativo	Cadena sin cerrar	23	14
7	?	Carácter no reconocido	28	46
8	&	Carácter no reconocido	34	24

Reporte de estadísticas

El sistema también genera un reporte de estadísticas basado en los resultados obtenidos durante el análisis.

Este reporte permite presentar información resumida sobre el proceso realizado.

Reporte de Estadísticas	
Resumen general del análisis léxico de PromptZal	
84 Total de Tokens	35 Total de Líneas
8 Total de Errores	
Tipo de Token	Cantidad
DIRECTIVA	3
RESERVADA	10
COMANDO_IA	6
FUNCION	1
CONECTOR	12
IDENTIFICADOR	27
LITERAL_CADENA	9
LITERAL_ENTERO	2
LITERAL_DECIMAL	0
OPERADOR_ASIGNACION	5
OPERADOR_CONCATENACION	1
DELIMITADOR	8
Reporte generado por el Analizador Léxico PromptZal	

Los reportes son generados a partir de la información obtenida directamente por el analizador, por lo que los resultados corresponden al contenido procesado por la aplicación.